

Лабораторна робота № 10

з курсу "Архітектура обчислювальних систем та комп'ютерна схемотехніка"

Виконав:

Студент групи ПМі-12

Козій Дем'ян

Тема: Логічні операції

Мета роботи: Використовуючи середовище Visual Studio та асемблер NASM, освоїти методи програмування з використанням логічних операцій на мові Assembler

Варіант: 11

Хід виконання роботи:

1. У середовищі Visual Studio було створено проект на мові C++ для реалізації навчального прикладу
2. З використанням асемблерної вставки `__asm` (синтаксис MASM) реалізовано програму для підрахунку в масиві з восьми чотирибітових слів кількості слів з парним числом одиниць у слові. Використано логічні операції `and`, `test`, `shl`, а також команди умовних переходів `jz`, `jnz`, `jnp`
3. Для виконання індивідуального завдання (варіант 11) було створено автономну 64-бітну програму на асемблері NASM
4. У програмі на NASM реалізовано послідовне введення 7 байтів (цілих чисел) користувачем у циклі `.input_loop` з використанням зовнішньої C-функції `scanf`
5. Зчитані байти об'єднуються у єдиний 64-бітний регістр `rax` за допомогою побітового зсуву вліво `shl` та логічного АБО `or` у циклі `.pack_loop`
6. Для аналізу масиву як 8 семибітних слів застосовано цикл `.analyze_loop`. За допомогою логічного множення (маска `and rdx, 01111111b`) виділяється поточне 7-бітне слово
7. Перевірка на непарну кількість нулів реалізована через команду `test dl, dl`, яка встановлює прапорець парності (Parity Flag). Оскільки непарна кількість нулів у 7-бітному слові тотожна парній кількості одиниць, використано команду умовного переходу `jpo .skip_increment`
8. Фінальний результат підрахунку виводиться на екран за допомогою зовнішньої функції `printf`

Програмний код:

1. Навчальний приклад (C++ з MASM-вставкою):

```
#include <iostream>
using namespace std;

int main() {
    int arr = 848843586;
    int res = 0;

    __asm {
        mov cx, 8
        mov eax, 1

    start_outer:
        mov bx, 4
        xor di, di

    start_inner:
        mov edx, arr
        and edx, eax
        jz go_next
        inc di

    go_next:
        shl eax, 1
        dec bx
        test bx, bx
        jnz start_inner
        and di, 1
        jnp end_
        inc res

    end_:
        loop start_outer
    }

    cout << res << endl;
    return(0);
}
```

2. Індивідуальне завдання (Варіант 11, NASM 64-bit):

```
1  default rel
2
3  extern printf
4  extern scanf
5  global main
6
7  section .data
8      fmt_in    db "%d", 0
9      fmt_out   db "%d", 10, 0
10
11 section .bss
12     array      resb 7
13     temp_input resd 1
14
15 section .text
16
17 main:
18     push rbx
19     sub rsp, 32
20
21     mov rbx, 0
22 .input_loop:
23     lea rcx, [fmt_in]
24     lea rdx, [temp_input]
25     call scanf
26
27     mov eax, [temp_input]
28     lea rcx, [array]
29     mov [rcx + rbx], al
30
31     inc rbx
32     cmp rbx, 7
33     jl .input_loop
```

```
35     xor rax, rax
36     mov rcx, 7
37     lea rsi, [array]
38
39     .pack_loop:
40         shl rax, 8
41         movzx rdx, byte [rsi]
42         or rax, rdx
43
44         inc rsi
45         dec rcx
46         jnz .pack_loop
47
48         mov rcx, 8
49         xor rbx, rbx
50
51     .analyze_loop:
52         mov rdx, rax
53         and rdx, 01111111b
54
55         test dl, dl
56         jpo .skip_increment
57
58         inc rbx
59
```

```
60     .skip_increment:
61         shr rax, 7
62         dec rcx
63         jnz .analyze_loop
64
65         lea rcx, [fmt_out]
66         mov rdx, rbx
67         call printf
68
69         add rsp, 32
70         pop rbx
71         xor eax, eax
72         ret
73
```

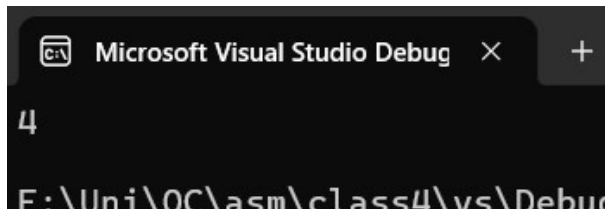
Результати виконання програми:

Для перевірки коректності роботи було проведено тестування обох програм.

Результат роботи прикладу (C++ MASM):

Вхідне значення жорстко задано в коді (848843586)

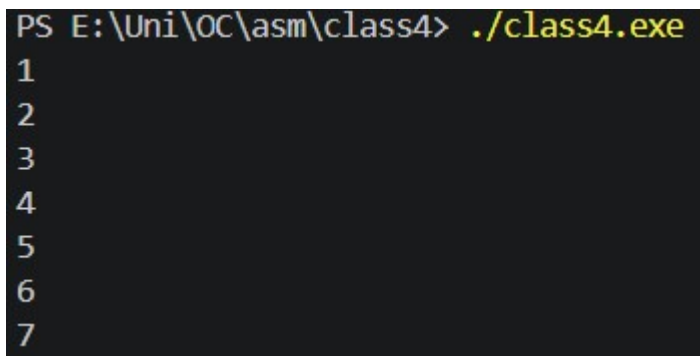
Вивід консолі:



Результат роботи індивідуального завдання (NASM):

Програма очікує почергове введення 7 цілих чисел (в діапазоні одного байта)

Введення (по одному числу через Enter):



Вивід консолі (кількість слів з непарним числом нулів):



Висновок:

Під час виконання лабораторної роботи № 10 було успішно засвоєно використання логічних операцій (AND, OR, TEST) та операцій зсуву (SHL, SHR) на мові Assembler. Практично досліджено два підходи до написання низькорівневого коду: використання асемблерних вставок (MASM) у середовищі C++ та створення автономної 64-бітної програми за допомогою NASM. В індивідуальному завданні успішно реалізовано алгоритм підрахунку слів з непарним числом нулів, ефективно використавши прапорець парності (Parity

Flag) та команду jro для оптимізації аналізу бітів. Це дозволило виконати умови завдання повністю та поглибити розуміння роботи процесора з бітовими масками.